

Inner Product Spaces and Similarity Transform for Pointing Detection

A.M. Krus

October 16, 2025

1 Introduction

This project explores how to determine the pointing direction of a gun-like controller equipped with an infrared (IR) camera. The objective is to map points detected in the camera frame to their corresponding positions on a display. While classical approaches use a full homography, our project uses a *similarity transform* instead, which preserves distances up to a uniform scale and rotation.

The mathematical context is best understood through the framework of inner product spaces, which provide notions of angle, length, distance, and orthogonal projection. Sections 2 and 3 provide the linear algebraic foundations. Later sections explore the concrete application of similarity matrices in place of homographies. References are provided for conceptual explanations.

2 Inner Product Spaces

Let $V = \mathbb{R}^n$ be a real vector space. An *inner product* is a function $\langle \cdot, \cdot \rangle : V \times V \rightarrow \mathbb{R}$ satisfying linearity, symmetry, and positivity. For $V = \mathbb{R}^3$ with the standard Euclidean inner product,

$$\langle \mathbf{x}, \mathbf{y} \rangle = x_1y_1 + x_2y_2 + x_3y_3.$$

From the inner product we derive:

- the induced norm: $\|\mathbf{x}\| = \sqrt{\langle \mathbf{x}, \mathbf{x} \rangle}$,
- the distance: $d(\mathbf{x}, \mathbf{y}) = \|\mathbf{x} - \mathbf{y}\|$,
- the angle: $\cos \theta = \frac{\langle \mathbf{x}, \mathbf{y} \rangle}{\|\mathbf{x}\| \|\mathbf{y}\|}$.

These notions allow us to interpret pixels as rays, compare pointing directions via angles, and measure reprojection errors via distances.

3 Orthogonal Projection

A fundamental operation in an inner product space is the *orthogonal projection*. Given a vector $\mathbf{x} \in \mathbb{R}^3$ and a unit vector $\mathbf{u}$, the projection of $\mathbf{x}$ onto the line spanned by $\mathbf{u}$ is

$$\text{proj}_{\mathbf{u}}(\mathbf{x}) = \frac{\langle \mathbf{x}, \mathbf{u} \rangle}{\langle \mathbf{u}, \mathbf{u} \rangle} \mathbf{u}.$$

For projection onto a plane with orthonormal basis $\{\mathbf{u}, \mathbf{v}\}$,

$$\text{proj}_{\text{span}\{\mathbf{u}, \mathbf{v}\}}(\mathbf{x}) = UU^T \mathbf{x}, \quad U = [\mathbf{u} \ \mathbf{v}].$$

In the context of this project:

- each pixel (u, v) corresponds to a ray $\mathbf{r}(u, v) \in \mathbb{R}^3$,
- the screen is modeled as a 2D plane in $\mathbb{R}^3$,
- finding where a ray intersects the screen can be viewed as projecting the ray onto the plane of the screen.

Thus, orthogonal projection is the geometric mechanism connecting camera rays to screen coordinates.

4 Camera Rays in $\mathbb{R}^3$

In the pinhole camera model, each pixel (u, v) corresponds to a normalized ray:

$$\mathbf{r}(u, v) = \frac{1}{\sqrt{x^2 + y^2 + 1}} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}, \quad x = \frac{u - c_x}{f_x}, \quad y = \frac{v - c_y}{f_y}.$$

5 Similarity Transform as a Linear Operator

The display surface is planar and can be parameterized by coordinates (X, Y) . Given a set of calibration points on the screen and their detected counterparts (u, v) in the camera, the projective homography can be replaced with a *similarity transform*

$$\lambda \begin{bmatrix} u \\ v \\ 1 \end{bmatrix} = S \begin{bmatrix} X \\ Y \\ 1 \end{bmatrix},$$

where

$$S = \begin{bmatrix} s \cos \theta & -s \sin \theta & t_x \\ s \sin \theta & s \cos \theta & t_y \\ 0 & 0 & 1 \end{bmatrix}.$$

The unknowns are scale s , rotation θ , and translation (t_x, t_y) .

6 Similarity Transform Estimation

6.1 Mathematical Formulation

Let $\mathbf{p}_i = (X_i, Y_i)^T$ be the world points and $\mathbf{q}_i = (u_i, v_i)^T$ their corresponding camera points. Expanding the similarity transform, we have

$$\begin{aligned} u_i &= s(\cos \theta X_i - \sin \theta Y_i) + t_x, \\ v_i &= s(\sin \theta X_i + \cos \theta Y_i) + t_y. \end{aligned}$$

Stacking $N \geq 2$ correspondences gives a linear system in four unknowns:

$$\begin{bmatrix} X_1 & -Y_1 & 1 & 0 \\ Y_1 & X_1 & 0 & 1 \\ \vdots & \vdots & \vdots & \vdots \\ X_N & -Y_N & 1 & 0 \\ Y_N & X_N & 0 & 1 \end{bmatrix} \begin{bmatrix} s \cos \theta \\ s \sin \theta \\ t_x \\ t_y \end{bmatrix} = \begin{bmatrix} u_1 \\ v_1 \\ \vdots \\ u_N \\ v_N \end{bmatrix}.$$

For $N = 3$, the system is exactly determined; for $N = 2$, we solve it in a least-squares sense.

6.2 Case Overview and Pseudocode

Listing 1: 4-point homography estimation

```
function computeHomography4( src [4][2] , dst [4][2] ) :
    build 8x8 system A, b from correspondences
    if solve8x8(A, b) succeeds:
        fill H matrix from solution
        return H
    else:
        return error
```

3 Points: Similarity Transform.

$$\begin{aligned} s &= \frac{\|\mathbf{q}_2 - \mathbf{q}_1\|}{\|\mathbf{p}_2 - \mathbf{p}_1\|}, \\ \theta &= \angle(\mathbf{q}_2 - \mathbf{q}_1, \mathbf{p}_2 - \mathbf{p}_1), \\ \mathbf{t} &= \mathbf{q}_1 - sR(\theta)\mathbf{p}_1, \quad R(\theta) = \begin{bmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{bmatrix}. \end{aligned}$$

Listing 2: 3-point similarity transform

```
function computeSimilarity3(src[3][2], dst[3][2]):
    compute s, theta from first two correspondences
    compute translation t from one correspondence
    construct S = [[sR, t], [0,0,1]]
    return S
```

2 Points: Approximate Similarity Transform.

$$s = \frac{\|\mathbf{q}_2 - \mathbf{q}_1\|}{\|\mathbf{p}_2 - \mathbf{p}_1\|},$$

$$\theta = \angle(\mathbf{q}_2 - \mathbf{q}_1, \mathbf{p}_2 - \mathbf{p}_1),$$

$$\mathbf{t} = \mathbf{q}_1 - sR(\theta)\mathbf{p}_1.$$

Listing 3: 2-point similarity transform estimation

```
function computeSimilarity2(src[2][2], dst[2][2]):
    dp = src[1] - src[0]
    dq = dst[1] - dst[0]
    s = norm(dq) / norm(dp)
    theta = atan2(dq.y, dq.x) - atan2(dp.y, dp.x)
    R = [[cos(theta), -sin(theta)], [sin(theta), cos(theta)]]
    t = dst[0] - s * R * src[0]
    construct S = [[sR, t], [0,0,1]]
    return S
```

6.3 Remarks on Least Squares Estimation

When more than three correspondences are available or measurements contain noise, we solve

$$\min_{\mathbf{x}} \|\mathbf{A}\mathbf{x} - \mathbf{b}\|^2, \quad \mathbf{x} = [s \cos \theta, s \sin \theta, t_x, t_y]^T.$$

This produces a statistically optimal estimate under Gaussian noise and ensures stable mapping of camera detections to screen coordinates.

7 Implementation Pipeline

1. Detect IR blobs in the camera image.
2. Associate them with known screen coordinates.
3. Build and solve the linear system for S .
4. Use S to map new detections (u, v) into screen coordinates.
5. Quantify error via induced norm in $\mathbb{R}^2$.

8 Conclusion

The inner product space framework provides the mathematical context to describe camera geometry in terms of rays, angles, and distances. Orthogonal projection offers the geometric link between rays and the screen plane. Using a similarity transform instead of a full homography ensures stable, low-dimensional mapping while preserving rotation, scale, and translation, which is sufficient for the pointing detection application.

References

- [1] Wikipedia: *Gaussian Elimination*. https://en.wikipedia.org/wiki/Gaussian_elimination
- [2] OpenCV: *Homography Tutorial*. https://docs.opencv.org/4.x/d9/dab/tutorial_homography.html
- [3] R. Hartley and A. Zisserman, *Multiple View Geometry in Computer Vision*, Cambridge University Press, 2nd ed., 2003.